TP 1 : Arbres de décision

Apprentissage supervisé

1 Préparation des données

Dans l'ensemble des TPs, il vous sera demandé de naviguer dans la documentation des différentes librairies pour obtenir des informations concernant les méthodes, paramètres et descriptions des objets référencés. Ces bibliothèques seront par exemple `NumPy`, `Pandas` ou encore `Scikit-Learn`. Cette dernière est accessible à l'adresse `https://scikit-learn.org/stable/api/index.html`.

Exercice 1.1: Chargez le jeu de données Iris avec la commande suivante :

```
from sklearn.datasets import load_iris
```

Insérez le dans un `DataFrame Pandas` et familiarisez vous avec les méthodes `head` et `info`.

```
iris = load_iris()

dataset = pd.DataFrame(data=..., columns=...)
dataset['species'] = pd.Categorical.from_codes(iris.target, iris.target_names)

print(dataset)
print(dataset.head())
print(dataset.info())
```

Réponse

```
iris = load_iris()

dataset = pd.DataFrame(data=iris.data, columns=iris.feature_names)
dataset['species'] = pd.Categorical.from_codes(iris.target, iris.target_names)

print(dataset)
print(dataset.head())
print(dataset.info())
```

```
     sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)    species
0                  5.1               3.5                1.4               0.2     setosa
1                  4.9               3.0                1.4               0.2     setosa
2                  4.7               3.2                1.3               0.2     setosa
3                  4.6               3.1                1.5               0.2     setosa
4                  5.0               3.6                1.4               0.2     setosa
..                 ...               ...                ...               ...        ...
145                6.7               3.0                5.2               2.3  virginica
146                6.3               2.5                5.0               1.9  virginica
147                6.5               3.0                5.2               2.0  virginica
148                6.2               3.4                5.4               2.3  virginica
149                5.9               3.0                5.1               1.8  virginica

[150 rows x 5 columns]

   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm) species
0                5.1               3.5                1.4               0.2  setosa
```

```
1                4.9               3.0                1.4               0.2  setosa
2                4.7               3.2                1.3               0.2  setosa
3                4.6               3.1                1.5               0.2  setosa
4                5.0               3.6                1.4               0.2  setosa

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
 4   species            150 non-null    category
dtypes: category(1), float64(4)
memory usage: 5.1 KB
None
```

Exercice 1.2: Décrivez chacune des variables du jeu de données, avec le type précis, la moyenne, la variance, le min et le max pour les variables quantitatives et le type précis ainsi que l'effectif de chaque valeur pour les variables qualitatives.

```python
# Variables quantitatives
print(dataset.describe())

# Variables qualitatives
counts = dataset['species'].value_counts()
print(counts)
```

Réponse

Le jeu de données contient quatre variables quantitatives continues et une variable qualitative nominale.

```
       sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
count         150.000000        150.000000         150.000000        150.000000
mean            5.843333          3.057333           3.758000          1.199333
std             0.828066          0.435866           1.765298          0.762238
min             4.300000          2.000000           1.000000          0.100000
25%             5.100000          2.800000           1.600000          0.300000
50%             5.800000          3.000000           4.350000          1.300000
75%             6.400000          3.300000           5.100000          1.800000
max             7.900000          4.400000           6.900000          2.500000


    species
setosa        50
versicolor    50
virginica     50
Name: count, dtype: int64
```

Exercice 1.3: Affichez l'histogramme de fréquence pour chacune des variables quantitatives du jeu de données. Vous pourrez vous aider de la bibliothèque `matplotlib.pyplot` et de la fonction `hist()`.

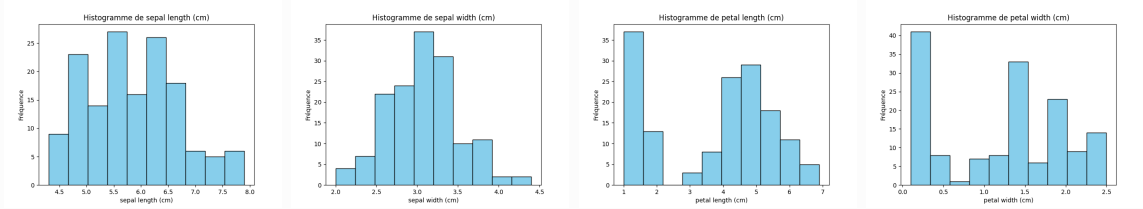
```python
import matplotlib.pyplot as plt

plt.hist(...)
plt.xlabel(...)
plt.ylabel(...)
plt.title(...)
plt.show()
```

Réponse

```
import matplotlib.pyplot as plt

for col in dataset.columns[:4]:
    plt.hist(dataset[col], bins=10, color='skyblue', edgecolor='black')
    plt.xlabel(col)
    plt.ylabel("Frequence")
    plt.title("Histogramme de " + col)
    plt.show()
```



Exercice 1.4: La fonction `pairplot` de `seaborn` permet de représenter les relations deux à deux entre toutes les variables du jeu de données. Affichez le `pairplot` correspondant au jeu de données Iris.

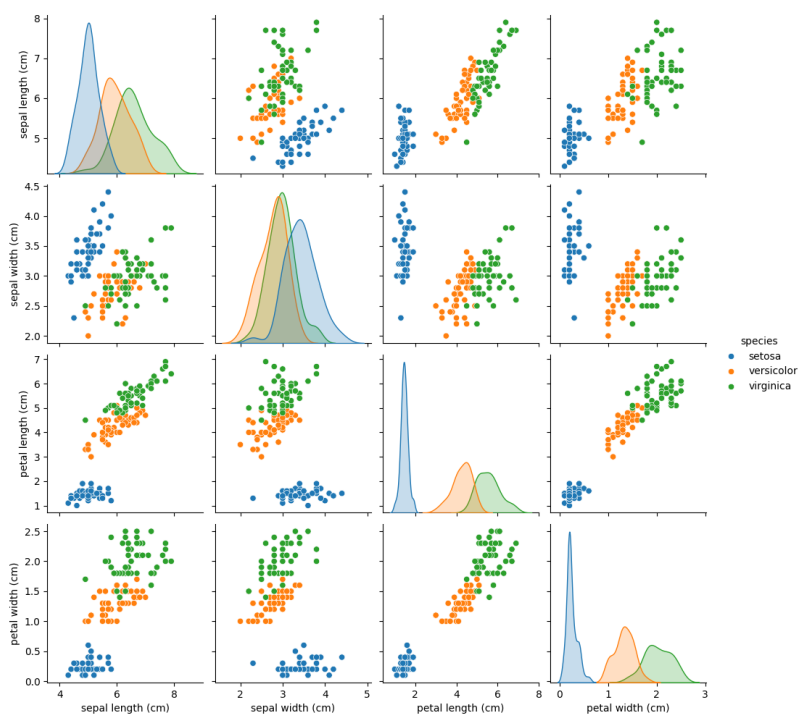
```
import seaborn as sns

sns.pairplot(..., hue=..., diag_kind='kde')
plt.suptitle(...)
plt.show()
```

Réponse

```
import seaborn as sns

sns.pairplot(dataset, hue='species', diag_kind='kde')
plt.suptitle("Scatter plots et densites du jeu Iris", y=1.02)
plt.show()
```



2 Arbre de décision

Dans cette partie, vous essayerez de prédire l'espèce d'une Iris (Setosa, Versicolor ou Virginica) en fonction de la longueur et la largeur de ses pétales, grâce à un arbre de décision.

Exercice 2.1: Créez un jeu de données à 2 dimensions, en ne retenant que la longueur et la largeur des pétales. Affichez ensuite le nuage de points grâce à la fonction `scatter` de `pyplot`.

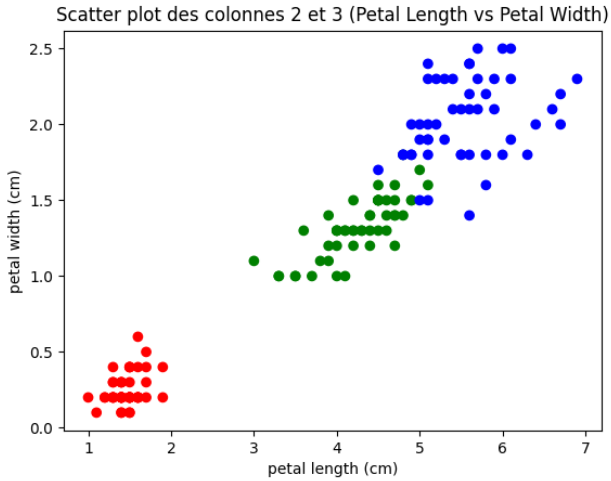
```
dataset_petal = dataset.iloc[:, [2,3]]
```

Réponse

```
dataset_petal = dataset.iloc[:, [2,3]]

colors = dataset['species'].map({'setosa':'red', 'versicolor':'green',
↪  'virginica':'blue'})

plt.scatter(dataset_petal.iloc[:, 0], dataset_petal.iloc[:, 1], c=colors)
plt.title("Scatter plot des colonnes 2 et 3 (Petal Length vs Petal Width)")
plt.xlabel(dataset_petal.columns[0])
plt.ylabel(dataset_petal.columns[1])
plt.show()
```



Exercice 2.2: Centrez à présent les données et affichez à nouveau le nuage de points.

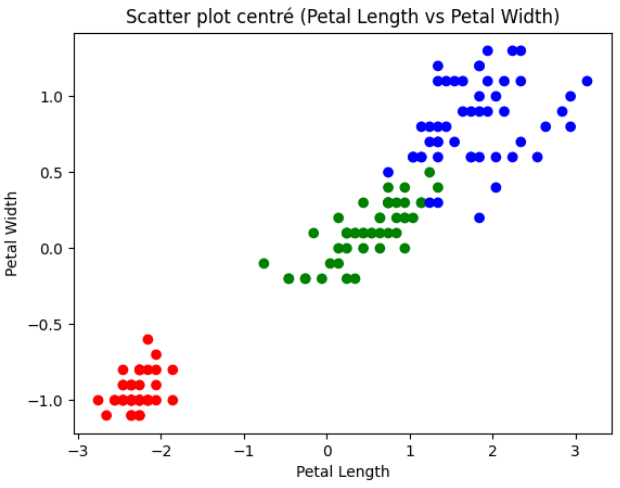
Réponse
Pour centrer les données on soustrait sa moyenne à chaque variable :

$$x'_j = x_j - \bar{x_j}. \tag{1}$$

```
petal_length = np.array(dataset_petal.iloc[:, 0])
petal_width = np.array(dataset_petal.iloc[:, 1])

petal_length = (petal_length - np.mean(petal_length))
petal_width = (petal_width - np.mean(petal_width))

plt.scatter(petal_length, petal_width, c=colors)
plt.title("Scatter plot centre (Petal Length vs Petal Width)")
plt.xlabel("Petal Length")
plt.ylabel("Petal Width")
plt.show()
```



Exercice 2.3: Calculez l'inertie totale du nuage de points, sachant que l'inertie mesure

la dispersion au centre de gravité :

$$\mathcal{I} = \frac{1}{n}\sum_{i=1}^{n} ||x_i - \bar{x}||^2. \tag{2}$$

Réponse
L'inertie peut s'écrire comme la somme des variances pour chaque dimensions :

$$\mathcal{I} = \sum_{j=1}^{p} \sigma_j^2, \tag{3}$$

avec σ_j^2 la variance empirique de la variable associée à l'indice j.

```
# Formule avec la variance
inertie = np.var(petal_length) + np.var(petal_width)
print(inertie)

# Formule avec la trace
X = np.array([petal_length, petal_width]).T
n = X.shape[0]
inertie = (1 / n) * np.sum(np.diag(X.T @ X))
print(inertie)
```

```
3.672635555555556
3.672635555555556
```

Exercice 2.4: (A faire à la fin du TP, pensez à regarder la correction) Montrez que lorsque les données sont centrées, l'inertie peut s'écrire en fonction de la trace du produit matriciel de la transposée de la matrice des données par elle même :

$$\mathcal{I} = \frac{1}{n} Tr(X^T X) = \sum_{j=1}^{p} \sigma_j^2. \tag{4}$$

Réponse

$$\begin{aligned} \mathcal{I} &= \frac{1}{n}\sum_{i=1}^{n} ||x_i - \bar{x}||^2 \\ &= \frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{p} (x_{ij} - \bar{x_j})^2 \\ &= \sum_{j=1}^{p} \sigma_j^2. \end{aligned} \tag{5}$$

Les variables étant centrées, on a $\bar{x_j} = 0, \forall j \in \{1, ..., p\}$:

$$\begin{aligned} \mathcal{I} &= \frac{1}{n}\sum_{i=1}^{n} ||x_i||^2 \\ &= \frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{p} x_{ij}^2 \\ &= \frac{1}{n} Tr(X^T X). \end{aligned} \tag{6}$$

Exercice 2.5: Calculez le coefficient de corrélation linéaire entre les deux variables étudiées. Vérifiez le résultat obtenu avec la fonction `corrcoef` de `numpy`.
Le coefficient de corrélation linéaire $r_{jj'}$ entre deux variables s'écrit :

$$r_{jj'} = \frac{cov(x_j, x_{j'})}{\sigma_j \sigma'_j}, \tag{7}$$

avec $cov(x_j, x_{j'}) = \frac{1}{n} x_j^T x_{j'}$.

Réponse

```
cov = (petal_length.T @ petal_width) / n
corr = cov / (np.std(petal_length) * np.std(petal_width))
print(corr)

print(np.corrcoef(petal_length, petal_width))
```

```
0.9628654314027961
[[1.         0.96286543]
 [0.96286543 1.        ]]
```

Exercice 2.6: (À faire en fin de TP) Montrez que le coefficient de corrélation linéaire entre deux variables centrées et réduites vaut toujours $\frac{1}{n}\langle x_j, x_{j'} \rangle$.

Réponse
Pour des variables centrées et réduites on a $\sigma_j = \sigma'_j = 1$ et donc :

$$r_{jj'} = cov(x_j, x_{j'}) = \frac{1}{n}\langle x_j, x_{j'} \rangle. \tag{8}$$

Exercice 2.7: Rappelons que pour entraîner un arbre de décision, les données n'ont pas besoin d'être centrées et réduites. (Elles seront tout de même centrées pour cet exercice, et nous utilisons les variables "Petal Width" et "Petal Length").

Pour commencer, créez une fonction `impurete(p)` qui prend en paramètres un vecteur de probabilités et qui retourne l'indice de Gini associé.
Vérifiez le résultat avec les trois vecteurs suivants :

```
p_test = np.array([
    [0.5, 0.5],
    [0, 1],
    [0.3, 0.7]
```

```
])

[print(impurete(p)) for p in p_test]
```

```
0.5
0.0
0.42000000000000004
```

Réponse
L'indice de Gini $G(p)$ associé au vecteur de probabilités p se calcule avec :

$$G(p) = 1 - \sum_{k=1}^{K} p_k^2. \tag{9}$$

```
def impurete(p):
    return 1 - np.sum(p**2)
```

Exercice 2.8: On s'intéresse maintenant au gain associé à la séparation d'un nœud S selon la variable d'indice j au seuil v. Avec la classe suivante :

```
class Noeud:
    def __init__(self, X, y, classes):
        self.X = np.array(X)
        self.y = np.array(y)
        self.n, self.p = self.X.shape
        self.classes = classes
```

`X` représente la matrice des données, de dimension (n, p), `y` représente le vecteur des classes, de dimension n et `classes` regroupe l'ensemble des $K = 3$ classes possibles, soit un vecteur de taille K. Définissez une fonction `gain(S, j, v)` qui retourne le gain associé.
Testez avec $j = 1$ (en informatique, les tableaux commencent généralement à l'indice 0, nous prendrons donc $j = 0$) et $v = -1$:

```
petal_length = np.array(dataset.iloc[:, 2])
petal_width = np.array(dataset.iloc[:, 3])

petal_length = (petal_length - np.mean(petal_length))
petal_width = (petal_width - np.mean(petal_width))

X = np.array([petal_length, petal_width]).T

S = Noeud(X, dataset['species'], np.unique(dataset['species']))
print(gain(S, 0, -1))
```

```
0.3333333333333334
```

On rappelle la formule du gain :

$$Gain(S, j, v) = G(p) - \left[\frac{n^-}{n} G(p^-) + \frac{n^+}{n} G(p^+)\right]. \tag{10}$$

```python
def gain(S, j, v):

    def proportions(S):
        return np.array([np.sum(S.y == classe) / S.n for classe in S.classes])

    n = S.X.shape[0]
    n_gauche = ...
    n_droite = ...

    p = proportions(S)
    p_gauche = proportions(...)
    p_droite = proportions(...)

    return ...
```

Réponse

```python
class Noeud:
    def __init__(self, X, y, classes):
        self.X = np.array(X)
        self.y = np.array(y)
        self.n, self.p = self.X.shape
        self.classes = classes

def gain(S, j, v):

    def proportions(S):
        return np.array([np.sum(S.y == classe) / S.n for classe in S.classes])

    n = S.X.shape[0]
    n_gauche = np.sum(S.X[:, j] <= v)
    n_droite = np.sum(S.X[:, j] > v)

    p = proportions(S)
    p_gauche = proportions(Noeud(S.X[S.X[:, j] <= v], S.y[S.X[:, j] <= v],
    ↪  S.classes))
    p_droite = proportions(Noeud(S.X[S.X[:, j] > v], S.y[S.X[:, j] > v],
    ↪  S.classes))

    return impurete(p) - (n_gauche/n) * impurete(p_gauche) - (n_droite/n) *
    ↪  impurete(p_droite)

S = Noeud(X, dataset['species'], np.unique(dataset['species']))
print(gain(S, 0, -1))
```

```
0.3333333333333334
```

Exercice 2.9: Écrivez la fonctions `optimise_gain(S)` qui retourne la variable associée à la meilleure séparation, son seuil et le gain associé. Testez avec le nœud S précédent, La réponse attendue est $(0, -1.31, 0.34)$.

```python
def optimise_gain(S):

    ...
    for j in range(S.X.shape[1]):
```

```
        valeurs = ...
        for n in range(len(valeurs) - 1):

            v = (valeurs[n] + valeurs[n + 1]) / 2

            gain_actuel = gain(...)
            if gain_actuel > gain_max:
                ...

    return ...

print(optimise_gain(S))
```

Réponse

```
def optimise_gain(S):

    gain_max = 0
    j_etoile = 0
    v_etoile = 0
    for j in range(S.X.shape[1]):

        valeurs = np.sort(list(set(S.X[:,j]))) # On enleve les doublons pour
        ↪  aller plus vite
        for n in range(len(valeurs) - 1):

            v = (valeurs[n] + valeurs[n + 1]) / 2

            gain_actuel = gain(S, j, v)
            if gain_actuel > gain_max:
                gain_max = gain_actuel
                j_etoile = j
                v_etoile = v

    return j_etoile, v_etoile, gain_max

print(optimise_gain(S))
```

```
(0, -1.31, 0.34)
```

Exercice 2.10: Afin de créer l'arbre, l'objet `Noeud` est enrichi comme suit :

```
class Noeud:
    def __init__(self, X, y, classes, j=0, v=0, gauche=None, droite=None):
        self.X = np.array(X)
        self.y = np.array(y)
        self.n, self.p = self.X.shape
        self.classes = classes
        self.j = j
        self.v = v
        self.gauche=gauche
        self.droite=droite
```

Créez une fonction récursive `arbre_recursif(S)` qui retourne la racine de l'arbre construit en ajoutant à chaque séparation, la variable associée $S.j$, le seuil $S.v$ et les nœuds enfants

S.gauche et *S.droite*.
Vous pouvez vous aider de :

```
def arbre_recursif(S):

    S.j, S.v, gain = ...

    if gain == 0:
        return S

    S.gauche = Noeud(S.X[S.X[:, S.j] <= S.v], S.y[S.X[:, S.j] <= S.v], S.classes)
    S.droite = ...

    arbre_recursif(...)
    arbre_recursif(...)

    return S
```

Testez avec le code suivant :

```
def print_tree(S, depth=0):
    indent = "  " * depth
    if S.gauche is None:
        y_list = list(S.y)
        print(f"{indent}Leaf: class={max(set(y_list), key=y_list.count)}")
    else:
        print(f"{indent}Feature[{S.j}] <= {S.v}")
        print_tree(S.gauche, depth+1)
        print_tree(S.droite, depth+1)

print_tree(S)
```

qui devrait retourner :

```
Feature[0] <= -1.3080000000000005
  Leaf: class=setosa
  Feature[1] <= 0.5506666666666664
    Feature[0] <= 1.1919999999999997
      Feature[1] <= 0.45066666666666644
        Leaf: class=versicolor
        Leaf: class=virginica
      Feature[1] <= 0.35066666666666646
        Leaf: class=virginica
        Feature[0] <= 1.6919999999999993
          Leaf: class=versicolor
          Leaf: class=virginica
    Feature[0] <= 1.0919999999999996
      Leaf: class=virginica
      Leaf: class=virginica
```

Réponse

```
class Noeud:
    def __init__(self, X, y, classes, j=0, v=0, gauche=None, droite=None):
        self.X = np.array(X)
        self.y = np.array(y)
        self.n, self.p = self.X.shape
        self.classes = classes
        self.j = j
        self.v = v
```

```
        self.gauche=gauche
        self.droite=droite

S = Noeud(X, dataset['species'], np.unique(dataset['species']))

def arbre_recursif(S):

    S.j, S.v, gain = optimise_gain(S)

    if gain == 0:
        return S

    S.gauche = Noeud(S.X[S.X[:, S.j] <= S.v], S.y[S.X[:, S.j] <= S.v], S.classes)
    S.droite = Noeud(S.X[S.X[:, S.j] > S.v], S.y[S.X[:, S.j] > S.v], S.classes)

    arbre_recursif(S.gauche)
    arbre_recursif(S.droite)

    return S

arbre_recursif(S)

def print_tree(S, depth=0):
    indent = "  " * depth
    if S.gauche is None:
        y_list = list(S.y)
        print(f"{indent}Leaf: class={max(set(y_list), key=y_list.count)}")
    else:
        print(f"{indent}Feature[{S.j}] <= {S.v}")
        print_tree(S.gauche, depth+1)
        print_tree(S.droite, depth+1)

print_tree(S)
```

Exercice 2.11: Écrivez maintenant une fonction `prediction(S, x)` qui prédit la classe de l'instance x avec l'arbre S, testez avec l'ensemble suivant :

```
X_test = [[-2, -0.7], #setosa
        [0.6, -0.1], #versicolor
        [1.7, 1]]   #virginica
```

Réponse

```
def prediction(S, x):
    if  S.gauche == None:
        return max(set(list(S.y)), key=list(S.y).count)

    if x[S.j] <= S.v:
        return prediction(S.gauche, x)
    else:
        return prediction(S.droite, x)

X_test = [[-2, -0.7],
        [0.6, -0.1],
        [1.7, 1]]
```

```
for x in X_test:
    print(prediction(S, x))
```

```
setosa
versicolor
virginica
```

Exercice 2.12: Afin de pouvoir réutiliser votre arbre sur n'importe quel jeu de données, créez dans un fichier `arbre.py` une classe `MonArbre` qui s'initialise avec un jeu de données (X, y) et possède possède une méthode `predict(self, X_test)` qui retourne la liste des prédictions associées aux instances de `X_test`.
Votre arbre doit pouvoir être appelé comme suit :

```
from arbre import MonArbre

X, y = load_iris(return_X_y = True)

X_test = [[5.5, 4, 1.5, 0.5],
        [5.5, 2, 4.5, 1.5],
        [7.5, 3, 7, 1.5]]

model = MonArbre(X, y)
predictions = model.predict(X_test)

print(predictions)
```

Réponse

```
import numpy as np

class Noeud:

    def __init__(self, X, y, classes, j=0, v=0, gauche=None, droite=None):
        self.X = np.array(X)
        self.y = np.array(y)
        self.n, self.p = self.X.shape
        self.classes = classes
        self.j = j
        self.v = v
        self.gauche=gauche
        self.droite=droite

class MonArbre:

    def __init__(self, X, y):
        self.racine = Noeud(X, y, np.unique(y))
        self.racine = self._train(self.racine)

    def predict(self, X_test):
        preds = []
        for x in X_test:
            preds.append(self._prediction(self.racine, x))

        return np.array(preds)
```

```
    def _prediction(self, S, x):
        if  S.gauche == None:
            return max(set(list(S.y)), key=list(S.y).count)

        if x[S.j] <= S.v:
            return self._prediction(S.gauche, x)
        else:
            return self._prediction(S.droite, x)

    def _train(self, S):
        S.j, S.v, gain = self._optimise_gain(S)

        if gain == 0:
            return S

        S.gauche = Noeud(S.X[S.X[:, S.j] <= S.v], S.y[S.X[:, S.j] <= S.v],
        ↪  S.classes)
        S.droite = Noeud(S.X[S.X[:, S.j] > S.v], S.y[S.X[:, S.j] > S.v],
        ↪  S.classes)

        self._train(S.gauche)
        self._train(S.droite)

        return S

    def _optimise_gain(self, S):
        gain_max = 0
        j_etoile = 0
        v_etoile = 0
        for j in range(S.X.shape[1]):

            valeurs = np.sort(list(set(S.X[:,j])))
            for n in range(len(valeurs) - 1):

                v = (valeurs[n] + valeurs[n + 1]) / 2

                gain_actuel = self._gain(S, j, v)
                if gain_actuel > gain_max:
                    gain_max = gain_actuel
                    j_etoile = j
                    v_etoile = v

        return j_etoile, v_etoile, gain_max

    def _purete(self, p):
        return 1 - np.sum(p**2)

    def _gain(self, S, j, v):

        def proportions(S):
            return np.array([np.sum(S.y == classe) / S.n for classe in
            ↪  S.classes])

        n = S.X.shape[0]
        n_gauche = np.sum(S.X[:, j] <= v)
        n_droite = np.sum(S.X[:, j] > v)
```

```
        p = proportions(S)
        p_gauche = proportions(Noeud(S.X[S.X[:, j] <= v], S.y[S.X[:, j] <= v],
        ↪  S.classes))
        p_droite = proportions(Noeud(S.X[S.X[:, j] > v], S.y[S.X[:, j] > v],
        ↪  S.classes))

        return self._purete(p) - (n_gauche/n) * self._purete(p_gauche) -
        ↪  (n_droite/n) * self._purete(p_droite)
```

3 La classe DecisionTreeClassifier

Exercice 3.1: La bibliothèque `scikit-learn` propose une classe permettant d'instancier et d'entraîner un arbre de décision.
Pour entraîner le modèle il faudra utiliser la méthode `fit()` comme suit :

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier()
model.fit(X, y)
```

Vérifiez avec la méthode `predict()` de cette classe, que les prédictions sont les mêmes qu'avec `MonArbre`.

Réponse

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier()
model.fit(X, y)

preds = model.predict(X_test)

if np.sum(preds == predictions) == preds.shape[0]:
    print("Predictions identiques.")
else:
    print("Predictions differentes.")
```

```
Prédictions identiques.
```

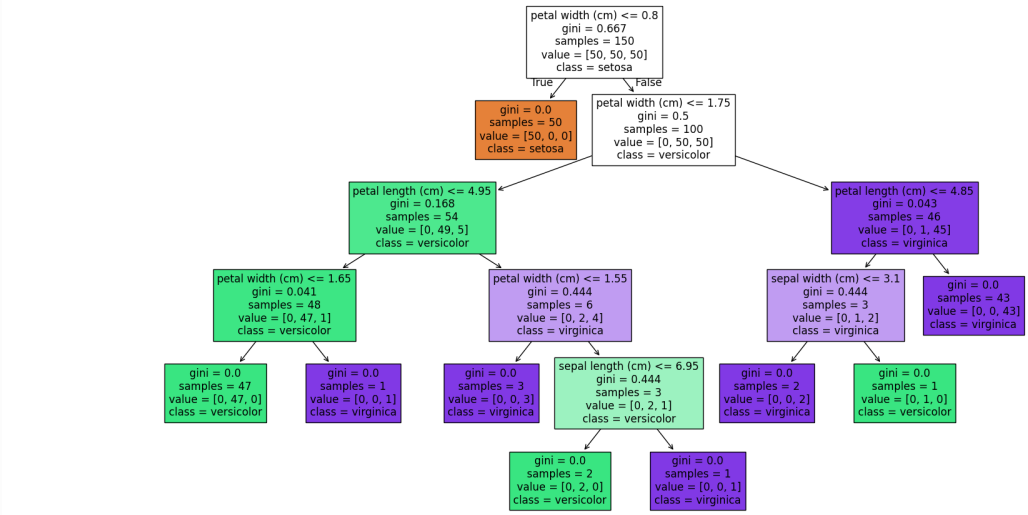
Exercice 3.2: Affichez votre arbre avec le code suivant :

```
from sklearn.tree import plot_tree

plt.figure(figsize=(20,50))
plot_tree(model, feature_names=iris.feature_names, class_names=iris.target_names,
↪  filled=True)
plt.show()
```

Quelles sont les trois feuilles les plus denses ? Quelle est leur indice de Gini ?

Réponse



Exercice 3.3: Pour testez les performances de notre arbre, nous allons séparer le jeu de données en jeu d'entraînement et jeu de test. La méthode `train_test_split` de `scikit-learn` permet de réaliser cette séparation en précisant la taille du jeu de test souhaité :

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
```

Entraînez votre modèle sur le jeu d'entrâinement et testez ses performances sur le jeu de test à l'aide de la méthode `score(X_test, y_test)` de la classe `DecisionTreeClassifier`. Relancez plusieurs fois votre code, que remarquez-vous et pourquoi ? Le score d'exactitude (*accuracy*) est obtenu avec la formule suivante :

$$accuracy = \frac{\text{nombre de prédictions correctes}}{\text{nombre total de prédictions}} \tag{11}$$

Réponse

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

model = DecisionTreeClassifier()
model.fit(X_train, y_train)

score = model.score(X_test, y_test)
print("Accuracy:", score)
```

```
Accuracy: 0.8888888888888888
```

Exercice 3.4: Répétez 100 fois aléatoirement l'étape d'entraînement, sur une séparation du jeu de données différente afin d'avoir une moyenne des performances du modèle sur le jeu de données Iris.

Réponse

```python
score = 0
for _ in range(100):
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

    model = DecisionTreeClassifier()
    model.fit(X_train, y_train)

    score += model.score(X_test, y_test)

print("Accuracy moyenne:", score / 100)
```

```
Accuracy moyenne: 0.9515555555555539
```

Exercice 3.5: La classe `DecisionTreeClassifier` offre aussi plusieurs critères d'élagage pré-entraînement avec les paramètres `max_depths`, qui régule la profondeur maximale de l'arbre lors de sa croissance, et `min_samples_leaf`, qui limite le nombre minimum d'éléments dans une feuille.
Vérifiez si cette régularisation permet d'obtenir de meilleures performances en affichant un graphique de l'évolution des performances moyennes du modèle en fonction de la valeur du paramètre d'élagage. D'abord avec :

```python
profondeur_max = [1, 3, 5, 8, 10, 12, 15]
```

Puis avec :

```python
densite_min = [1, 2, 3, 4, 5, 6, 7]
```

Réponse

```python
profondeur_max = [1, 3, 5, 8, 10, 12, 15]
scores = []

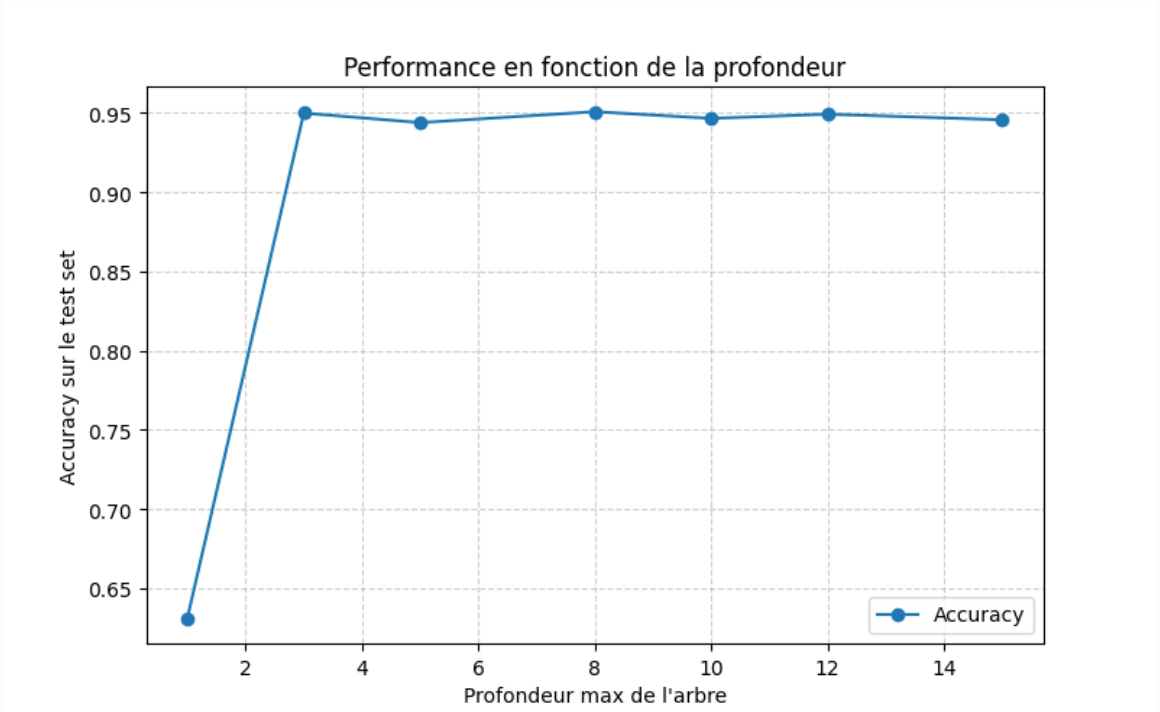
for profondeur in profondeur_max:
    score = 0
    for _ in range(100):
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

        model = DecisionTreeClassifier(max_depth=profondeur)
        model.fit(X_train, y_train)

        score += model.score(X_test, y_test)
    scores.append(score / 100)

plt.figure(figsize=(8, 5))
plt.plot(profondeur_max, scores, marker="o", linestyle="-", label="Accuracy")
plt.xlabel("Profondeur max de l'arbre")
```

```
plt.ylabel("Accuracy sur le test set")
plt.title("Performance en fonction de la profondeur")
plt.grid(True, linestyle="--", alpha=0.6)
plt.legend()
plt.show()
```



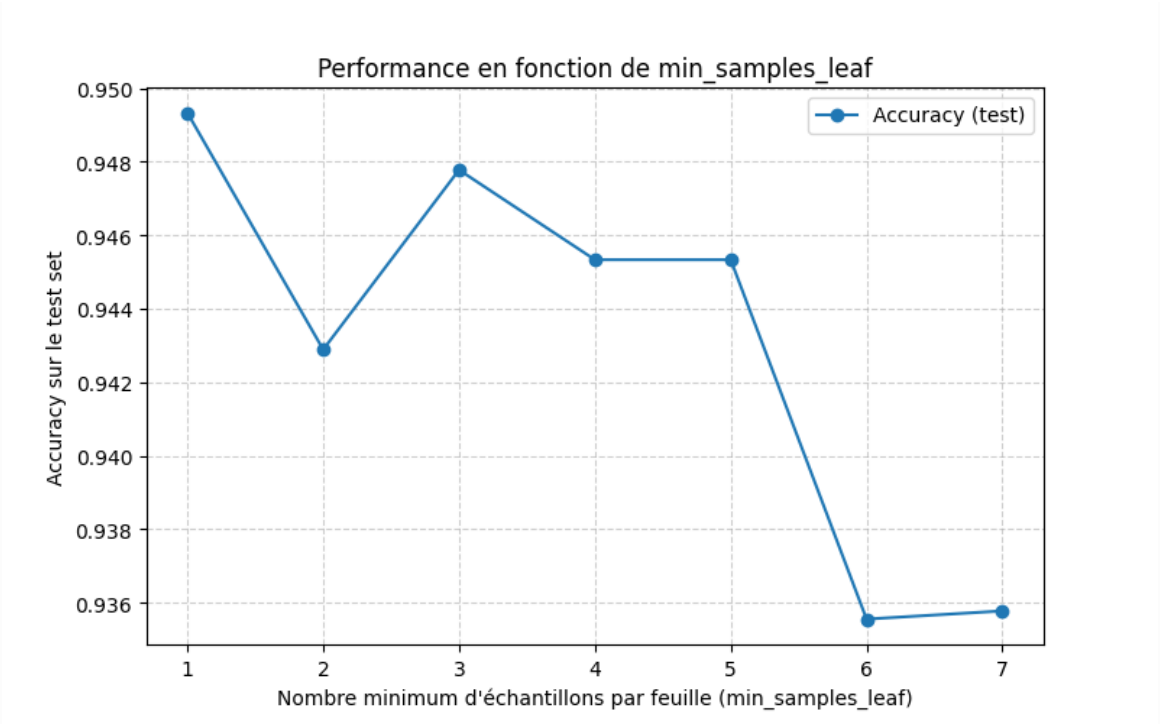

```
densite_min = [1, 2, 3, 4, 5, 6, 7]
scores = []

for densite in densite_min:
    score = 0
    for _ in range(100):
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

        model = DecisionTreeClassifier(min_samples_leaf=densite)
        model.fit(X_train, y_train)

        score += model.score(X_test, y_test)
    scores.append(score / 100)

plt.figure(figsize=(8, 5))
plt.plot(densite_min, scores, marker="o", linestyle="-", label="Accuracy (test)")
plt.xlabel("Nombre minimum d'echantillons par feuille (min_samples_leaf)")
plt.ylabel("Accuracy sur le test set")
plt.title("Performance en fonction de min_samples_leaf")
plt.grid(True, linestyle="--", alpha=0.6)
plt.legend()
plt.show()
```

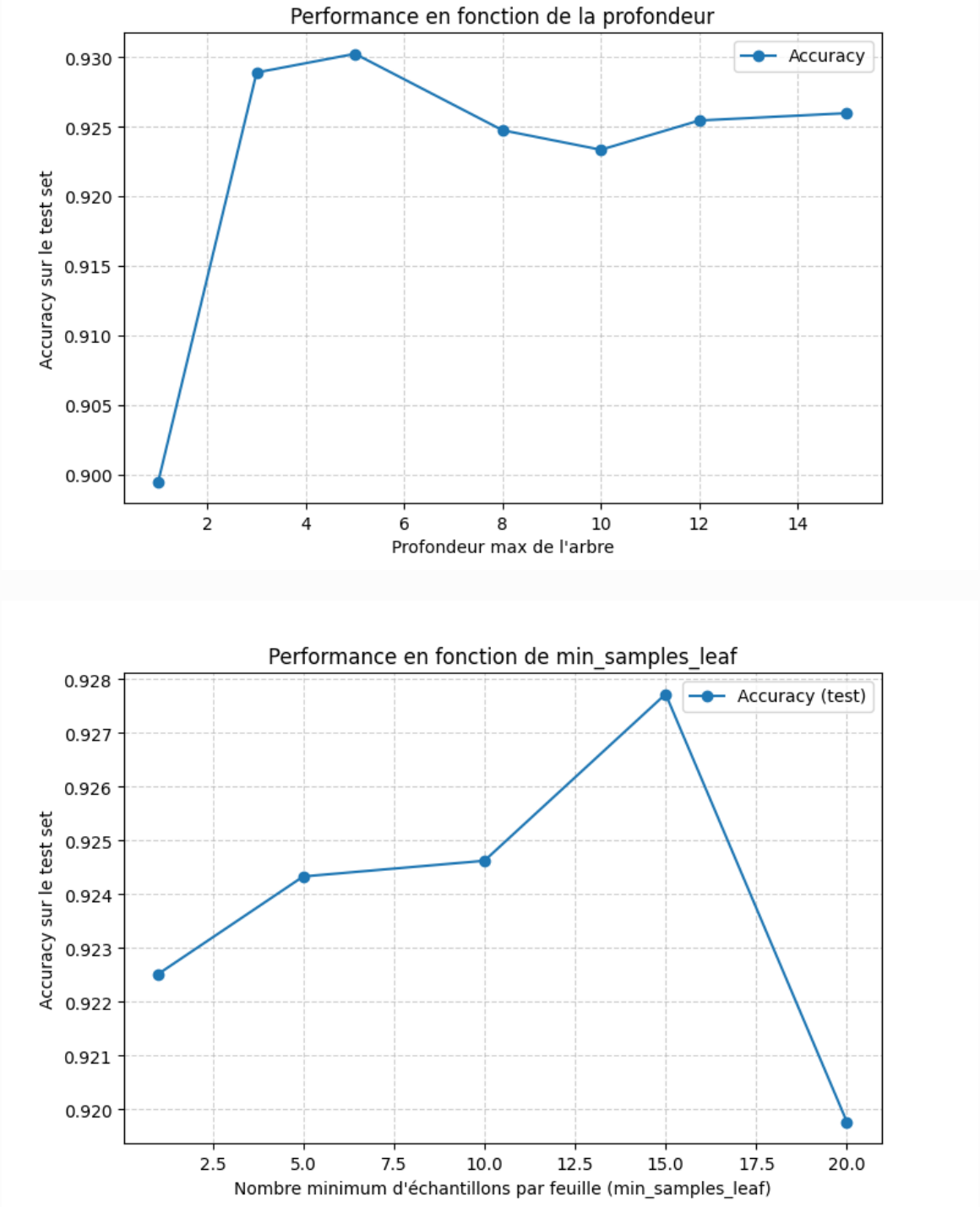


Le jeu de données Iris étant extrèmement simple, le modèle n'a pas de mal à généraliser (pas de sur-apprentissage) et une régularisation ne permets pas d'augmenter les performancees sur modèle.

Exercice 3.6: Pour étudier le comportement de cet élagage, répétez la dernière étape sur un autre jeu de données plus complexe et vérifiez l'effet de la régularisation.

```
from sklearn.datasets import load_breast_cancer
X, y = load_breast_cancer(return_X_y = True)
```

Réponse



Le jeu de données Breast Cancer étant plus complexe, la régularisation permets au modèle de gagner en performance.

4 MonArbre élagué

Exercice 4.1: Implémentez l'élagage par pronfodeur maximale et par densité de feuille minimale dans la classe `MonArbre`. Rendez également compatible le modèle avec les variables qualitatives et avec la régression.